

Chapter 7 — Sorting-Based Patterns

0. What You Will Learn in This Chapter

Sorting is one of the most powerful tools in coding interviews.

Many problems look difficult only because the input is unordered. Once you sort the data, relationships become clearer:

- duplicates become adjacent
- smallest and largest values move to the ends
- intervals can be processed in order
- greedy choices become easier
- two pointers become possible
- overlap checks become simpler
- custom orderings become possible

By the end of this chapter, you should understand:

- Why sorting helps
- When to sort
- When not to sort
- How sorting enables scanning
- How sorting enables two pointers
- How sorting enables greedy
- How to sort intervals
- How to handle duplicate values after sorting
- How to write C++ comparators
- What mistakes to avoid with sorting
- How to reason about $O(n \log n)$ complexity

Core LeetCode problems covered in this chapter:

- 1.1. Contains Duplicate, sorting alternative
 - 2.2. Valid Anagram, sorting alternative
 - 3.3. Merge Intervals
 - 4.4. Insert Interval
 - 5.5. Non-overlapping Intervals
 - 6.6. Meeting Rooms, premium-style
 - 7.7. Meeting Rooms II
 - 8.8. Minimum Number of Arrows to Burst Balloons
 - 9.9. 3Sum
 - 10.10. 3Sum Closest
 - 11.11. 4Sum
 - 12.12. K Closest Points to Origin
 - 13.13. Largest Number
 - 14.14. Assign Cookies
 - 15.15. Boats to Save People
 - 16.16. Queue Reconstruction by Height
 - 17.17. Reorder Data in Log Files
-

1. What Is a Sorting-Based Pattern?

A sorting-based pattern is any problem where sorting the input makes the solution significantly easier.

Example:

```
nums = {5, 1, 3, 2, 4}
```

After sorting:

```
nums = {1, 2, 3, 4, 5}
```

Now:

- smallest is at the front
- largest is at the back
- close values are near each other
- duplicates are adjacent
- two-pointer movement becomes meaningful

Sorting does not solve every problem by itself, but it often transforms the problem into a simpler pattern:

```
Sort + scan  
Sort + two pointers  
Sort + greedy  
Sort + interval merge  
Sort + heap  
Sort + custom comparator
```

2. Why Sorting Exists as a Pattern

Unordered data hides structure.

Example:

```
intervals = {{8,10}, {1,3}, {2,6}, {15,18}}
```

It is not immediately clear which intervals overlap.

After sorting by start time:

```
{{1,3}, {2,6}, {8,10}, {15,18}}
```

Now overlapping intervals are next to each other.

You can scan once and merge.

So sorting exists as a pattern because it creates useful order.

3. Core Need / Why Use Sorting Over Other Patterns?

Sorting is useful when the problem becomes easier after ordering the data.

3.1 Sorting Makes Duplicates Adjacent

Example:

```
nums = {3, 1, 2, 3, 4}
```

After sorting:

```
{1, 2, 3, 3, 4}
```

Duplicate 3s are adjacent.

Now you can detect duplicates with neighbor comparison.

3.2 Sorting Enables Two Pointers

For pair/triplet sum problems, sorting gives direction.

If array is sorted:

```
nums[left] + nums[right]
```

If sum is too small, move left.

If sum is too large, move right.

Without sorting, this movement is not meaningful.

3.3 Sorting Helps Intervals

Intervals are much easier when sorted by:

- start time, for merging
- end time, for greedy scheduling

Example:

Merge intervals:

Sort by start.

Non-overlapping intervals:

Sort by end.

3.4 Sorting Enables Greedy Choices

Some greedy problems require picking the “smallest useful” or “earliest ending” item. Sorting makes that possible.

Examples:

- Assign Cookies
- Boats to Save People
- Non-overlapping Intervals
- Minimum Number of Arrows to Burst Balloons

3.5 Sorting Enables Custom Ordering

Sometimes normal ascending order is not enough.

Example:

Largest Number:

```
nums = {3, 30, 34, 5, 9}
```

Normal descending numeric order does not work.

You need a custom comparator:

```
a + b > b + a
```

4. When to Use Sorting

Use sorting when you see these clues.

4.1 Duplicates / Uniqueness

Problem asks:

- contains duplicate
- remove duplicates
- unique triplets
- group similar values
- repeated elements

Examples:

- Contains Duplicate
 - 3Sum
 - 4Sum
 - Subsets II, later backtracking
-

4.2 Intervals

Problem gives intervals:

[start, end]

and asks about:

- merge
- overlap
- meeting rooms
- minimum removals
- arrows
- free time

Examples:

- Merge Intervals
 - Insert Interval
 - Meeting Rooms
 - Non-overlapping Intervals
 - Minimum Number of Arrows
-

4.3 Pair / Triplet / Quadruplet Sum

Problem asks:

- find pair
- find triplets
- find quadruplets
- closest sum
- unique combinations

Examples:

- Two Sum II
 - 3Sum
 - 3Sum Closest
 - 4Sum
-

4.4 Greedy Assignment

Problem asks:

- assign resources
- minimize number of groups/boats/arrows

- maximize satisfied children
- schedule intervals

Examples:

- Assign Cookies
 - Boats to Save People
 - Non-overlapping Intervals
 - Queue Reconstruction by Height
-

4.5 Closest / Minimum Difference

Sorting places close values near each other.

Examples:

- Minimum Absolute Difference
 - K Closest Elements
 - Heaters
-

4.6 Custom Comparator

Problem asks:

- sort by distance
- sort by frequency
- sort by end time
- sort by special string rule
- form largest/smallest concatenation

Examples:

- K Closest Points
 - Largest Number
 - Reorder Data in Log Files
-

5. When NOT to Sort

Sorting is powerful, but it can be wrong.

5.1 Original Order Matters

Example:

Best Time to Buy and Sell Stock.

`prices = {7, 1, 5, 3, 6, 4}`

You cannot sort prices, because days matter.

Wrong:

`sort(prices.begin(), prices.end());`

Sorting destroys time order.

5.2 Original Indices Are Required

Example:

LeetCode 1 Two Sum asks for original indices.

If you sort, you lose original positions unless you store them.

Hash map is usually cleaner.

5.3 Subarray Problems

Subarray means contiguous in original order.

Sorting destroys contiguity.

Example:

Maximum Subarray cannot be solved by sorting.

5.4 Subsequence Problems

Subsequence preserves relative order.

Sorting destroys that order.

Example:

Longest Increasing Subsequence should not be solved by sorting the original array directly.

5.5 $O(n)$ Solution Exists and Sorting Is Too Slow

Example:

Contains Duplicate can be solved in $O(n)$ average with hash set.

Sorting is $O(n \log n)$.

Sorting may still pass, but it is not optimal.

6. Sorting Complexity

In C++:

```
sort(nums.begin(), nums.end());
```

Time:

```
 $O(n \log n)$ 
```

Space:

Usually $O(\log n)$ stack space internally.

In interviews, you can say:

```
Time:  $O(n \log n)$ 
```

```
Space:  $O(\log n)$  or  $O(1)$  extra depending on implementation
```

But if you create an output array, include that space separately.

7. C++ Sorting Tools Needed

7.1 Basic Ascending Sort

```
sort(nums.begin(), nums.end());
```

Example:

```
vector<int> nums = {5, 2, 1};  
sort(nums.begin(), nums.end());  
// nums = {1, 2, 5}
```

7.2 Descending Sort

```
sort(nums.begin(), nums.end(), greater<int>());
```

Example:

```
vector<int> nums = {5, 2, 1};  
sort(nums.begin(), nums.end(), greater<int>());  
// nums = {5, 2, 1}
```

7.3 Sorting Pairs

```
vector<pair<int, int>> arr;  
sort(arr.begin(), arr.end());
```

Default:

- sort by first
- if tie, sort by second

Example:

```
{{2,3}, {1,5}, {1,2}}
```

becomes:

```
{{1,2}, {1,5}, {2,3}}
```

7.4 Sorting Vector of Vectors

For:

```
vector<vector<int>> intervals;
```

Default sort is lexicographic.

```
sort(intervals.begin(), intervals.end());
```

For intervals [start, end], this sorts by start, then end.

7.5 Custom Comparator

Sort intervals by end:

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    return a[1] < b[1];  
});
```

Use references to avoid copying:

```
const vector<int>& a
```

7.6 Comparator With Tie-Breaker

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    if (a[1] != b[1]) {
```

```
        return a[1] < b[1];
    }

    return a[0] < b[0];
});
```

7.7 Strict Weak Ordering Rule

Comparator must return true only if a should come before b.

Do not write:

```
return a[0] <= b[0];
```

This is wrong.

Use:

```
return a[0] < b[0];
```

Why?

If values are equal, both `comp(a,b)` and `comp(b,a)` should not be true.

8. Core Templates

Template 1 — Sort and Scan

```
sort(nums.begin(), nums.end());

for (int i = 0; i < nums.size(); i++) {
    // process sorted nums[i]
}
```

Used for:

- duplicates
- minimum difference
- grouping adjacent values

Template 2 — Sort Intervals by Start

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {
    return a[0] < b[0];
});
```

Used for:

- Merge Intervals
- Meeting Rooms
- Insert Interval, if not already sorted

Template 3 — Sort Intervals by End

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    return a[1] < b[1];  
});
```

Used for:

- Non-overlapping Intervals
- Minimum Number of Arrows
- Activity selection

Template 4 — Merge Intervals

```
vector<vector<int>> mergeIntervals(vector<vector<int>>& intervals) {  
    if (intervals.empty()) {  
        return {};  
    }  
  
    sort(intervals.begin(), intervals.end());  
  
    vector<vector<int>> result;  
    vector<int> current = intervals[0];  
  
    for (int i = 1; i < intervals.size(); i++) {  
        if (intervals[i][0] <= current[1]) {  
            current[1] = max(current[1], intervals[i][1]);  
        } else {  
            result.push_back(current);  
            current = intervals[i];  
        }  
    }  
  
    result.push_back(current);  
  
    return result;  
}
```

Template 5 — Greedy by Earliest End

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    return a[1] < b[1];  
});  
  
int count = 0;
```

```

int lastEnd = INT_MIN;

for (auto& interval : intervals) {
    if (interval[0] >= lastEnd) {
        count++;
        lastEnd = interval[1];
    }
}

```

Used when you want maximum number of non-overlapping intervals.

Template 6 — Remove Overlapping Intervals

```

sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {
    return a[1] < b[1];
});

int removals = 0;
int lastEnd = intervals[0][1];

for (int i = 1; i < intervals.size(); i++) {
    if (intervals[i][0] < lastEnd) {
        removals++;
    } else {
        lastEnd = intervals[i][1];
    }
}

```

Template 7 — Sort + Two Pointers

```

sort(nums.begin(), nums.end());

int left = 0;
int right = nums.size() - 1;

while (left < right) {
    int sum = nums[left] + nums[right];

    if (sum == target) {
        // found
        left++;
        right--;
    } else if (sum < target) {

```

```
        left++;
    } else {
        right--;
    }
}
```

Template 8 — Fixed Index + Two Pointers

```
sort(nums.begin(), nums.end());

for (int i = 0; i < nums.size(); i++) {
    if (i > 0 && nums[i] == nums[i - 1]) {
        continue;
    }

    int left = i + 1;
    int right = nums.size() - 1;

    while (left < right) {
        long long sum = (long long)nums[i] + nums[left] + nums[right];

        if (sum == target) {
            // store answer

            left++;
            right--;

            while (left < right && nums[left] == nums[left - 1]) {
                left++;
            }

            while (left < right && nums[right] == nums[right + 1]) {
                right--;
            }
        } else if (sum < target) {
            left++;
        } else {
            right--;
        }
    }
}
```

Template 9 — Sort by Distance

```
sort(points.begin(), points.end(), [](const vector<int>& a, const vector<int>& b) {  
    int distA = a[0] * a[0] + a[1] * a[1];  
    int distB = b[0] * b[0] + b[1] * b[1];  
  
    return distA < distB;  
});
```

Use squared distance to avoid square root.

Template 10 — Custom String Comparator

For Largest Number:

```
sort(words.begin(), words.end(), [](const string& a, const string& b) {  
    return a + b > b + a;  
});
```

9. Major Variations of Sorting-Based Problems

Sort and scan	adjacent relationships	Contains Duplicate
Sort intervals by start	merge/check overlap	Merge Intervals
Sort intervals by end	greedy scheduling	Non-overlapping Intervals
Sort + two pointers	pair/triplet sum	3Sum
Sort + heap	active resources	Meeting Rooms II
Sort + custom comparator	special ordering	Largest Number
Sort + greedy	local optimal after order	Assign Cookies
Sort + duplicate skipping	unique combinations	3Sum / 4Sum

10. Pattern Recognition Checklist

Ask:

2. Would ordering the data make relationships clearer?
3. Are duplicates important?

4. Are intervals involved?
5. Do I need to merge or detect overlap?
6. Do I need a pair/triplet/quadruplet sum?
7. Can I sort without destroying required order?
8. Is a greedy choice easier after sorting?
9. Do I need custom ordering?
10. Are closest values relevant?
11. Will sorting reduce nested comparisons?

If yes, sorting may be the key.

11. Full Problem Walkthrough 1 — Contains Duplicate Sorting Alternative

11.1 Problem Summary

Given an array `nums`, return `true` if any value appears at least twice.

Example:

```
nums = {1, 2, 3, 1}
```

Output:

```
true
```

11.2 How to Identify Sorting

Duplicates become adjacent after sorting.

This is not the most optimal solution compared to hashing, but it is a useful sorting pattern.

11.3 C++ Code

```
class Solution {
public:
    bool containsDuplicate(vector<int>& nums) {
        sort(nums.begin(), nums.end());

        for (int i = 1; i < nums.size(); i++) {
            if (nums[i] == nums[i - 1]) {
                return true;
            }
        }

        return false;
    }
};
```

11.4 Dry Run

```
nums = {1, 2, 3, 1}
```

Sorted:

```
{1, 1, 2, 3}
```

Compare adjacent:

```
nums[1] == nums[0]
```

Return true.

11.5 Complexity

Time: $O(n \log n)$

Space: $O(1)$ or $O(\log n)$, depending on sort implementation.

11.6 When This Is Useful

Use when:

- hash map space is undesirable
 - sorting is allowed
 - $O(n \log n)$ is acceptable
-

11.7 Similar Problems

- Valid Anagram sorting alternative
 - Find Duplicate Number, different constraints
 - Intersection of Two Arrays
-

12. Full Problem Walkthrough 2 — Valid Anagram Sorting Alternative

12.1 Problem Summary

Return true if two strings are anagrams.

Example:

```
s = "anagram"
```

```
t = "nagaram"
```

Output:

```
true
```

12.2 How Sorting Helps

Anagrams become identical after sorting characters.

```
"anagram" -> "aaagmnr"
```

```
"nagaram" -> "aaagmnr"
```

12.3 C++ Code

```
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.size() != t.size()) {
            return false;
        }

        sort(s.begin(), s.end());
        sort(t.begin(), t.end());

        return s == t;
    }
};
```

12.4 Complexity

Let n be string length.

Time: $O(n \log n)$

Space: depends on sorting implementation.

Frequency-counting solution is $O(n)$, but sorting is simple.

12.5 Similar Problems

- Group Anagrams
 - Find All Anagrams in a String
 - Permutation in String
-

13. Full Problem Walkthrough 3 — LeetCode 56: Merge Intervals

13.1 Problem Summary

Given intervals, merge all overlapping intervals.

Example:

```
intervals = {{1,3}, {2,6}, {8,10}, {15,18}}
```

Output:

```
{{1,6}, {8,10}, {15,18}}
```

13.2 How to Identify This Pattern

Problem says:

- intervals
- merge overlapping

Sort by start time.

After sorting, overlapping intervals appear next to each other.

13.3 Brute Force Idea

Compare every interval with every other interval and merge overlaps.
This is messy and can become $O(n^2)$.

13.4 Optimized Intuition

Sort by start.

Maintain current merged interval.

For each next interval:

- if it overlaps current, extend current
 - otherwise push current and start new one
-

13.5 C++ Code

```
class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        if (intervals.empty()) {
            return {};
        }

        sort(intervals.begin(), intervals.end());

        vector<vector<int>> result;
        vector<int> current = intervals[0];

        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] <= current[1]) {
                current[1] = max(current[1], intervals[i][1]);
            } else {
                result.push_back(current);
                current = intervals[i];
            }
        }

        result.push_back(current);

        return result;
    }
};
```

13.6 Code Explanation

```
sort(intervals.begin(), intervals.end());
```

Sorts by start time, then end time.

```
if (intervals[i][0] <= current[1])
```

Overlap condition for closed intervals.

```
current[1] = max(current[1], intervals[i][1]);
```

Extend merged interval.

13.7 Dry Run

```
intervals = {{1,3}, {2,6}, {8,10}, {15,18}}
```

Already sorted.

{2,6}	{1,3}	yes	current = {1,6}
{8,10}	{1,6}	no	push {1,6}, current = {8,10}
{15,18}	{8,10}	no	push {8,10}, current = {15,18}

Final push:

```
{15,18}
```

Output:

```
{{1,6}, {8,10}, {15,18}}
```

13.8 Correctness Explanation

After sorting by start time, if the next interval does not overlap the current merged interval, no later interval can overlap current either because later intervals start even later. So it is safe to finalize current.

13.9 Complexity

Time: $O(n \log n)$

Space: $O(n)$ for output.

13.10 Edge Cases

- empty input
- one interval
- touching intervals: {1,4}, {4,5}
- fully contained interval
- all intervals overlap
- no intervals overlap

13.11 Common Mistakes

Mistake 1: Forgetting final push

After loop, current interval still needs to be added.

Mistake 2: Not sorting

Without sorting, overlaps may be missed.

Mistake 3: Wrong overlap condition

For LeetCode 56, intervals are closed, so:

```
start <= currentEnd
```

means overlap.

13.12 Similar Problems

- Insert Interval
- Meeting Rooms
- Non-overlapping Intervals
- Employee Free Time

14. Full Problem Walkthrough 4 — LeetCode 57: Insert Interval

14.1 Problem Summary

You are given sorted non-overlapping intervals and a new interval.

Insert and merge if necessary.

Example:

```
intervals = {{1,3}, {6,9}}  
newInterval = {2,5}
```

Output:

```
{{1,5}, {6,9}}
```

14.2 How to Identify This Pattern

Problem gives:

- sorted intervals
- non-overlapping intervals
- insert one interval
- merge overlaps

Since input is already sorted, no sorting is needed.

This is interval scan.

14.3 Three-Part Intuition

There are three groups:

12. intervals completely before newInterval
13. intervals overlapping newInterval
14. intervals completely after newInterval

14.4 C++ Code

```

class Solution {
public:
    vector<vector<int>> insert(vector<vector<int>>& intervals, vector<int>&
newInterval) {
        vector<vector<int>> result;

        int i = 0;
        int n = intervals.size();

        while (i < n && intervals[i][1] < newInterval[0]) {
            result.push_back(intervals[i]);
            i++;
        }

        while (i < n && intervals[i][0] <= newInterval[1]) {
            newInterval[0] = min(newInterval[0], intervals[i][0]);
            newInterval[1] = max(newInterval[1], intervals[i][1]);
            i++;
        }

        result.push_back(newInterval);

        while (i < n) {
            result.push_back(intervals[i]);
            i++;
        }

        return result;
    }
};

```

14.5 Code Explanation

```
intervals[i][1] < newInterval[0]
```

Interval ends before new interval starts, so no overlap.

```
intervals[i][0] <= newInterval[1]
```

Interval starts before or at new interval end, so overlap.

Merge:

```
newInterval[0] = min(...)
```

```
newInterval[1] = max(...)
```

14.6 Dry Run

```
intervals = {{1,3}, {6,9}}  
newInterval = {2,5}
```

First interval {1,3} overlaps with {2,5}.
Merge:

```
newInterval = {1,5}
```

Then {6,9} is after.
Output:

```
{{1,5}, {6,9}}
```

14.7 Complexity

Time: $O(n)$
Space: $O(n)$ for output.
No sorting needed.

14.8 Edge Cases

- insert before all intervals
- insert after all intervals
- overlaps many intervals
- intervals empty
- new interval contained inside existing interval

14.9 Common Mistakes

- sorting unnecessarily
- using wrong non-overlap condition
- forgetting to push merged new interval

14.10 Similar Problems

- Merge Intervals
- Interval List Intersections
- Employee Free Time

15. Full Problem Walkthrough 5 — LeetCode 435: Non-overlapping Intervals

15.1 Problem Summary

Given intervals, return minimum number of intervals to remove so the rest are non-overlapping.
Example:

```
intervals = {{1,2}, {2,3}, {3,4}, {1,3}}
```

Output:

1

Remove {1,3}.

15.2 How to Identify This Pattern

Problem asks:

- minimum removals
- intervals non-overlapping

Equivalent:

Maximize number of intervals kept.

This is greedy by earliest end time.

Sort by end.

15.3 Greedy Intuition

Always keep the interval that ends earliest.

Why?

It leaves the most room for future intervals.

15.4 C++ Code

```
class Solution {
public:
    int eraseOverlapIntervals(vector<vector<int>>& intervals) {
        if (intervals.empty()) {
            return 0;
        }

        sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const
vector<int>& b) {
            return a[1] < b[1];
        });

        int removals = 0;
        int lastEnd = intervals[0][1];

        for (int i = 1; i < intervals.size(); i++) {
            if (intervals[i][0] < lastEnd) {
                removals++;
            } else {
                lastEnd = intervals[i][1];
            }
        }

        return removals;
    }
};
```

```
}  
};
```

15.5 Important Overlap Rule

For this problem:

```
{1,2}  
{2,3}
```

do not overlap.
So overlap is:

```
start < lastEnd
```

not:

```
start <= lastEnd
```

15.6 Dry Run

```
intervals = {{1,2}, {2,3}, {3,4}, {1,3}}
```

Sorted by end:

```
{{1,2}, {1,3}, {2,3}, {3,4}}
```

{1,3}	2	yes	remove
{2,3}	2	no	keep, lastEnd = 3
{3,4}	3	no	keep, lastEnd = 4

Removals = 1.

15.7 Correctness Explanation

Choosing the interval with earliest end is optimal because it leaves maximum remaining space. Any other overlapping interval ending later would reduce future choices.

15.8 Complexity

Time: $O(n \log n)$

Space: $O(1)$ extra, ignoring sort stack.

15.9 Edge Cases

- no overlap
- all overlap
- touching intervals
- same end time
- empty input

15.10 Similar Problems

- Minimum Number of Arrows
 - Maximum Length of Pair Chain
 - Meeting Rooms
 - Activity Selection
-

16. Full Problem Walkthrough 6 — Meeting Rooms

This is a premium-style classic.

16.1 Problem Summary

Given meeting intervals, determine if a person can attend all meetings.

Example:

```
intervals = {{0,30}, {5,10}, {15,20}}
```

Output:

```
false
```

Because meetings overlap.

16.2 How to Identify This Pattern

Problem asks:

- intervals
- check overlap

Sort by start time and compare adjacent intervals.

16.3 C++ Code

```
bool canAttendMeetings(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end());

    for (int i = 1; i < intervals.size(); i++) {
        if (intervals[i][0] < intervals[i - 1][1]) {
            return false;
        }
    }

    return true;
}
```

16.4 Why Adjacent Comparison Works

After sorting by start time, if an interval overlaps any previous interval, it must overlap the immediately previous one with the latest nearby start.

16.5 Complexity

Time: $O(n \log n)$

Space: $O(1)$ extra.

16.6 Edge Cases

- no meetings
 - one meeting
 - touching meetings
 - all overlapping
 - sorted input already
-

16.7 Similar Problems

- Meeting Rooms II
 - Merge Intervals
 - Non-overlapping Intervals
-

17. Full Problem Walkthrough 7 — LeetCode 253: Meeting Rooms II

17.1 Problem Summary

Given meeting intervals, return minimum number of conference rooms required.

Example:

```
intervals = {{0,30}, {5,10}, {15,20}}
```

Output:

```
2
```

17.2 How to Identify This Pattern

Problem asks:

- intervals
- maximum overlapping meetings
- minimum rooms

Sort by start time and use min heap of end times.

17.3 Intuition

Process meetings in chronological start order.

Maintain a min heap of room end times.

For each meeting:

- if earliest ending room is free, reuse it
 - otherwise allocate new room
-

17.4 C++ Code

```
int minMeetingRooms(vector<vector<int>>& intervals) {  
    if (intervals.empty()) {
```

```

    return 0;
}

sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const
vector<int>& b) {
    return a[0] < b[0];
});

priority_queue<int, vector<int>, greater<int>> minHeap;

int rooms = 0;

for (auto& meeting : intervals) {
    int start = meeting[0];
    int end = meeting[1];

    if (!minHeap.empty() && minHeap.top() <= start) {
        minHeap.pop();
    }

    minHeap.push(end);

    rooms = max(rooms, (int)minHeap.size());
}

return rooms;
}

```

17.5 Dry Run

intervals = {{0,30}, {5,10}, {15,20}}

{0,30}	{}	add room ending 30	{30}	1
{5,10}	{30}	need new room	{10,30}	2
{15,20}	{10,30}	reuse room ending 10	{20,30}	2

Return 2.

17.6 Complexity

Time: $O(n \log n)$

Sorting plus heap operations.

Space: $O(n)$

17.7 Edge Cases

- no meetings
 - touching meetings
 - all meetings overlap
 - no meetings overlap
 - many same start times
-

17.8 Alternative: Sweep Line

```
int minMeetingRooms(vector<vector<int>>& intervals) {
    vector<int> starts;
    vector<int> ends;

    for (auto& interval : intervals) {
        starts.push_back(interval[0]);
        ends.push_back(interval[1]);
    }

    sort(starts.begin(), starts.end());
    sort(ends.begin(), ends.end());

    int s = 0;
    int e = 0;
    int rooms = 0;
    int best = 0;

    while (s < starts.size()) {
        if (starts[s] < ends[e]) {
            rooms++;
            best = max(best, rooms);
            s++;
        } else {
            rooms--;
            e++;
        }
    }

    return best;
}
```

17.9 Similar Problems

- Meeting Rooms

- Employee Free Time
 - Car Pooling
 - Minimum Number of Platforms
-

18. Full Problem Walkthrough 8 — LeetCode 452: Minimum Number of Arrows to Burst Balloons

18.1 Problem Summary

Each balloon is an interval [start, end].

An arrow shot at position x bursts every balloon where:

$\text{start} \leq x \leq \text{end}$

Return minimum arrows needed.

Example:

$\text{points} = \{\{10,16\}, \{2,8\}, \{1,6\}, \{7,12\}\}$

Output:

2

18.2 How to Identify This Pattern

Problem asks:

- intervals
- minimum arrows to cover intervals
- one arrow can cover overlapping intervals

Sort by end time.

Greedy: shoot arrow at earliest ending balloon.

18.3 C++ Code

```
class Solution {
public:
    int findMinArrowShots(vector<vector<int>>& points) {
        if (points.empty()) {
            return 0;
        }

        sort(points.begin(), points.end(), [](const vector<int>& a, const vector<int>& b) {
            return a[1] < b[1];
        });

        int arrows = 1;
```

```

    long long arrowPos = points[0][1];

    for (int i = 1; i < points.size(); i++) {
        if (points[i][0] > arrowPos) {
            arrows++;
            arrowPos = points[i][1];
        }
    }

    return arrows;
}
};

```

18.4 Dry Run

Sorted by end:

{{1,6}, {2,8}, {7,12}, {10,16}}

{1,6}	6	yes	1
{2,8}	6	yes	1
{7,12}	6	no	2
{10,16}	12	yes	2

Return 2.

18.5 Important Overlap Rule

If:

start == arrowPos

arrow still bursts balloon.

New arrow only if:

start > arrowPos

18.6 Complexity

Time: $O(n \log n)$

Space: $O(1)$ extra.

18.7 Similar Problems

- Non-overlapping Intervals
- Meeting Rooms
- Maximum Length of Pair Chain

19. Full Problem Walkthrough 9 —

LeetCode 15: 3Sum

19.1 Problem Summary

Return all unique triplets whose sum is zero.

Example:

```
nums = {-1, 0, 1, 2, -1, -4}
```

Output:

```
{{-1,-1,2}, {-1,0,1}}
```

19.2 How Sorting Helps

Sorting helps in two ways:

15. enables two pointers
16. makes duplicate skipping possible

19.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> threeSum(vector<int>& nums) {
        vector<vector<int>> result;

        sort(nums.begin(), nums.end());

        for (int i = 0; i < nums.size(); i++) {
            if (i > 0 && nums[i] == nums[i - 1]) {
                continue;
            }

            int left = i + 1;
            int right = nums.size() - 1;

            while (left < right) {
                long long sum = (long long)nums[i] + nums[left] + nums[right];

                if (sum == 0) {
                    result.push_back({nums[i], nums[left], nums[right]});

                    left++;
                    right--;
                }
            }
        }

        return result;
    }
};
```

```
        while (left < right && nums[left] == nums[left - 1]) {
            left++;
        }

        while (left < right && nums[right] == nums[right + 1]) {
            right--;
        }
        } else if (sum < 0) {
            left++;
        } else {
            right--;
        }
    }
}

return result;
}
};
```

19.4 Complexity

Time: $O(n^2)$

Space: $O(1)$ extra, ignoring output.

19.5 Common Mistakes

- not sorting
 - not skipping duplicates
 - reusing same index
 - overflow in sum
 - returning duplicate triplets
-

19.6 Similar Problems

- 3Sum Closest
 - 4Sum
 - 3Sum Smaller
-

20. Full Problem Walkthrough 10 — LeetCode 16: 3Sum Closest

20.1 Problem Summary

Given array `nums` and `target`, return sum of three integers closest to `target`.

Example:

```
nums = {-1, 2, 1, -4}
target = 1
```

Output:

2

Because $-1 + 2 + 1 = 2$.

20.2 How to Identify This Pattern

Problem asks:

- triplet
- closest sum
- sorted movement can guide improvement

Use sorting + fixed index + two pointers.

20.3 C++ Code

```
class Solution {
public:
    int threeSumClosest(vector<int>& nums, int target) {
        sort(nums.begin(), nums.end());

        int closest = nums[0] + nums[1] + nums[2];

        for (int i = 0; i < nums.size(); i++) {
            int left = i + 1;
            int right = nums.size() - 1;

            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];

                if (abs(sum - target) < abs(closest - target)) {
                    closest = sum;
                }

                if (sum < target) {
                    left++;
                } else if (sum > target) {
                    right--;
                } else {
                    return target;
                }
            }
        }
    }
}
```

```
        return closest;
    }
};
```

20.4 Dry Run

nums = {-1, 2, 1, -4}
target = 1

Sorted:

{-4, -1, 1, 2}

Try combinations with two pointers.
Closest becomes 2.

20.5 Complexity

Time: $O(n^2)$
Space: $O(1)$ extra.

20.6 Edge Cases

- exactly target
- negative target
- duplicates
- only 3 elements

20.7 Similar Problems

- 3Sum
- 4Sum
- K Sum variants

21. Full Problem Walkthrough 11 — LeetCode 18: 4Sum

21.1 Problem Summary

Return all unique quadruplets that sum to target.
Example:

nums = {1, 0, -1, 0, -2, 2}
target = 0

Output:

{{-2,-1,1,2}, {-2,0,0,2}, {-1,0,0,1}}

21.2 How to Identify This Pattern

Problem asks:

- quadruplets
- unique
- target sum

Sort, fix two indices, then use two pointers.

21.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> fourSum(vector<int>& nums, int target) {
        vector<vector<int>> result;

        sort(nums.begin(), nums.end());

        int n = nums.size();

        for (int i = 0; i < n; i++) {
            if (i > 0 && nums[i] == nums[i - 1]) {
                continue;
            }

            for (int j = i + 1; j < n; j++) {
                if (j > i + 1 && nums[j] == nums[j - 1]) {
                    continue;
                }

                int left = j + 1;
                int right = n - 1;

                while (left < right) {
                    long long sum = (long long)nums[i] + nums[j] + nums[left] +
nums[right];

                    if (sum == target) {
                        result.push_back({nums[i], nums[j], nums[left], nums[right]});

                        left++;
                        right--;

                        while (left < right && nums[left] == nums[left - 1]) {
                            left++;
                        }
                    }
                }
            }
        }

        return result;
    }
};
```

```

        while (left < right && nums[right] == nums[right + 1]) {
            right--;
        }
    } else if (sum < target) {
        left++;
    } else {
        right--;
    }
}
}
}

return result;
}
};

```

21.4 Complexity

Time: $O(n^3)$

Space: $O(1)$ extra, ignoring output.

21.5 Common Mistakes

- not using long long
- duplicate quadruplets
- wrong duplicate skip for second fixed index
- starting left too early

21.6 Similar Problems

- 3Sum
- 4Sum II, hashing
- K Sum generalization

22. Full Problem Walkthrough 12 — LeetCode 973: K Closest Points to Origin

22.1 Problem Summary

Given points on a 2D plane, return the k closest points to origin.

Example:

```

points = {{1,3}, {-2,2}}
k = 1

```

Output:

{{-2,2}}

22.2 How Sorting Helps

Sort points by squared distance:

$x^2 + y^2$

No need for square root.

22.3 C++ Code

```
class Solution {
public:
    vector<vector<int>> kClosest(vector<vector<int>>& points, int k) {
        sort(points.begin(), points.end(), [](const vector<int>& a, const vector<int>&
b) {
            int distA = a[0] * a[0] + a[1] * a[1];
            int distB = b[0] * b[0] + b[1] * b[1];

            return distA < distB;
        });

        return vector<vector<int>>(points.begin(), points.begin() + k);
    }
};
```

22.4 Complexity

Time: $O(n \log n)$

Space: $O(k)$ for output.

22.5 Alternatives

For better performance when k is small:

- heap: $O(n \log k)$
- quickselect: average $O(n)$

Sorting is easiest and often acceptable.

22.6 Similar Problems

- Kth Largest Element
- Top K Frequent Elements
- K Closest Elements

23. Full Problem Walkthrough 13 — LeetCode 179: Largest Number

23.1 Problem Summary

Given non-negative integers, arrange them to form the largest possible number.

Example:

```
nums = {3, 30, 34, 5, 9}
```

Output:

```
"9534330"
```

23.2 How to Identify This Pattern

Problem asks for special ordering.

Normal numeric sorting fails.

Need custom comparator.

For two strings a and b, decide order by comparing:

```
a + b
```

```
b + a
```

23.3 C++ Code

```
class Solution {
public:
    string largestNumber(vector<int>& nums) {
        vector<string> words;

        for (int x : nums) {
            words.push_back(to_string(x));
        }

        sort(words.begin(), words.end(), [](const string& a, const string& b) {
            return a + b > b + a;
        });

        if (words[0] == "0") {
            return "0";
        }

        string result;

        for (string& word : words) {
            result += word;
        }

        return result;
    }
};
```

```
}  
};
```

23.4 Why Comparator Works

For two numbers 3 and 30:

"3" + "30" = "330"

"30" + "3" = "303"

Since "330" is larger, "3" should come before "30".

23.5 Complexity

Let:

- n = number of elements
- m = average digit length

Time:

$O(n \log n * m)$

Space:

$O(n * m)$

23.6 Edge Cases

- all zeroes
- one number
- numbers with different digit lengths
- repeated numbers

23.7 Common Mistakes

- numeric descending sort
- forgetting all-zero case
- comparator using $\geq$

23.8 Similar Problems

- Reorder Data in Log Files
- Custom comparator problems
- Sort by concatenation logic

24. Full Problem Walkthrough 14 — LeetCode 455: Assign Cookies

24.1 Problem Summary

Each child has greed factor $g[i]$.

Each cookie has size $s[j]$.

A child is content if:

```
s[j] >= g[i]
```

Return maximum number of content children.

24.2 How to Identify This Pattern

Problem asks:

- assign resources
- maximize satisfied count
- smallest sufficient resource is best

Sort both arrays and greedily match.

24.3 C++ Code

```
class Solution {
public:
    int findContentChildren(vector<int>& g, vector<int>& s) {
        sort(g.begin(), g.end());
        sort(s.begin(), s.end());

        int child = 0;
        int cookie = 0;

        while (child < g.size() && cookie < s.size()) {
            if (s[cookie] >= g[child]) {
                child++;
            }

            cookie++;
        }

        return child;
    }
};
```

24.4 Intuition

Give the smallest possible cookie to the least greedy child.

If a cookie cannot satisfy the least greedy child, it cannot satisfy anyone else either, so skip it.

24.5 Complexity

Time: $O(n \log n + m \log m)$

Space: $O(1)$ extra.

24.6 Similar Problems

- Boats to Save People

- Candy
 - Queue Reconstruction by Height
-

25. Full Problem Walkthrough 15 — LeetCode 881: Boats to Save People

25.1 Problem Summary

Each boat can carry at most two people with total weight at most limit.
Return minimum boats.

Example:

```
people = {3, 2, 2, 1}
limit = 3
```

Output:

```
3
```

25.2 How Sorting Helps

Sort people by weight.

Pair lightest with heaviest if possible.

The heaviest remaining person must take a boat.

25.3 C++ Code

```
class Solution {
public:
    int numRescueBoats(vector<int>& people, int limit) {
        sort(people.begin(), people.end());

        int left = 0;
        int right = people.size() - 1;
        int boats = 0;

        while (left <= right) {
            if (people[left] + people[right] <= limit) {
                left++;
            }

            right--;
            boats++;
        }
    }
};
```

```
    return boats;
}
};
```

25.4 Correctness Intuition

The heaviest person must be placed. If the lightest can go with them, pair them. If even the lightest cannot fit, the heaviest must go alone.

25.5 Complexity

Time: $O(n \log n)$

Space: $O(1)$ extra.

25.6 Similar Problems

- Assign Cookies
- Two Sum II
- Container With Most Water

26. Full Problem Walkthrough 16 — LeetCode 406: Queue Reconstruction by Height

26.1 Problem Summary

Each person is represented as:

[h, k]

where:

- h = height
- k = number of people in front with height greater than or equal to h

Reconstruct the queue.

Example:

people = {{7,0}, {4,4}, {7,1}, {5,0}, {6,1}, {5,2}}

Output:

{{5,0}, {7,0}, {5,2}, {6,1}, {4,4}, {7,1}}

26.2 How to Identify This Pattern

This is a custom sort + greedy insertion problem.

Taller people affect shorter people's k.

Shorter people do not affect taller people's k.

So process taller people first.

26.3 Sorting Rule

Sort by:

17. height descending
18. k ascending

```
sort(people.begin(), people.end(), [](const vector<int>& a, const vector<int>& b) {  
    if (a[0] != b[0]) {  
        return a[0] > b[0];  
    }  
  
    return a[1] < b[1];  
});
```

Then insert each person at index k.

26.4 C++ Code

```
class Solution {  
public:  
    vector<vector<int>> reconstructQueue(vector<vector<int>>& people) {  
        sort(people.begin(), people.end(), [](const vector<int>& a, const vector<int>&  
b) {  
            if (a[0] != b[0]) {  
                return a[0] > b[0];  
            }  
  
            return a[1] < b[1];  
        });  
  
        vector<vector<int>> result;  
  
        for (auto& person : people) {  
            int index = person[1];  
            result.insert(result.begin() + index, person);  
        }  
  
        return result;  
    }  
};
```

26.5 Why This Works

When placing a person, all already placed people are taller or same height.
So inserting at index k guarantees exactly k taller-or-equal people before them.
Shorter people inserted later will not affect their count.

26.6 Complexity

Sorting:

$O(n \log n)$

Insertion into vector can cost $O(n)$.

Total:

$O(n^2)$

Space:

$O(n)$

26.7 Common Mistakes

- sorting height ascending
 - sorting same height by k descending
 - not understanding why taller first works
 - using `push_back` instead of insertion at k
-

26.8 Similar Problems

- Russian Doll Envelopes
 - Non-overlapping Intervals
 - Greedy ordering problems
-

27. Full Problem Walkthrough 17 — LeetCode 937: Reorder Data in Log Files

27.1 Problem Summary

Reorder logs so that:

19. letter-logs come before digit-logs
 20. letter-logs are sorted lexicographically by content
 21. if content ties, sort by identifier
 22. digit-logs remain in original order
-

27.2 How to Identify This Pattern

Problem asks for special sorting rules.

Use stable sort + custom comparator.

27.3 C++ Code

```
class Solution {
public:
    vector<string> reorderLogFiles(vector<string>& logs) {
        stable_sort(logs.begin(), logs.end(), [](const string& a, const string& b) {
            int posA = a.find(' ');
```

```

int posB = b.find(' ');

string idA = a.substr(0, posA);
string idB = b.substr(0, posB);

string contentA = a.substr(posA + 1);
string contentB = b.substr(posB + 1);

bool digitA = isdigit(contentA[0]);
bool digitB = isdigit(contentB[0]);

if (!digitA && !digitB) {
    if (contentA != contentB) {
        return contentA < contentB;
    }

    return idA < idB;
}

if (!digitA && digitB) {
    return true;
}

if (digitA && !digitB) {
    return false;
}

return false;
});

return logs;
}
};

```

27.4 Why Stable Sort?

Digit logs should remain in original order.
 When both are digit logs, comparator returns false.
 stable_sort preserves their original relative order.

27.5 Complexity

Let:

- n = number of logs
- m = average log length

Time:

$O(n \log n * m)$

Space depends on sorting and string operations.

27.6 Common Mistakes

- sorting digit logs
- ignoring identifier tie-breaker
- using normal sort instead of `stable_sort`
- incorrectly detecting digit logs

28. Mini Problem 1 — Minimum Absolute Difference

Pattern

Sort and compare adjacent values.

```
class Solution {
public:
    vector<vector<int>> minimumAbsDifference(vector<int>& arr) {
        sort(arr.begin(), arr.end());

        int best = INT_MAX;
        vector<vector<int>> result;

        for (int i = 1; i < arr.size(); i++) {
            int diff = arr[i] - arr[i - 1];

            if (diff < best) {
                best = diff;
                result.clear();
                result.push_back({arr[i - 1], arr[i]});
            } else if (diff == best) {
                result.push_back({arr[i - 1], arr[i]});
            }
        }

        return result;
    }
};
```

Why sorting?

Minimum absolute difference must occur between adjacent sorted values.

29. Mini Problem 2 — LeetCode 56

Alternative Logic

Instead of storing current, you can directly merge into result:

```
class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end());

        vector<vector<int>> result;

        for (auto& interval : intervals) {
            if (result.empty() || interval[0] > result.back()[1]) {
                result.push_back(interval);
            } else {
                result.back()[1] = max(result.back()[1], interval[1]);
            }
        }

        return result;
    }
};
```

This version is concise and common.

30. Mini Problem 3 — Sort Characters By Frequency

Pattern

Frequency map + sorting.

```
class Solution {
public:
    string frequencySort(string s) {
        unordered_map<char, int> freq;

        for (char c : s) {
            freq[c]++;
        }

        vector<pair<char, int>> arr;
```

```

    for (auto& p : freq) {
        arr.push_back({p.first, p.second});
    }

    sort(arr.begin(), arr.end(), [](const pair<char, int>& a, const pair<char, int>&
b) {
        return a.second > b.second;
    });

    string result;

    for (auto& p : arr) {
        result += string(p.second, p.first);
    }

    return result;
}
};

```

This combines hashing + sorting.

31. Mini Problem 4 — LeetCode 1288: Remove Covered Intervals

Pattern

Sort by start ascending, end descending.

```

class Solution {
public:
    int removeCoveredIntervals(vector<vector<int>>& intervals) {
        sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const
vector<int>& b) {
            if (a[0] != b[0]) {
                return a[0] < b[0];
            }

            return a[1] > b[1];
        });

        int count = 0;
        int maxEnd = 0;

        for (auto& interval : intervals) {

```

```
        if (interval[1] > maxEnd) {  
            count++;  
            maxEnd = interval[1];  
        }  
    }  
  
    return count;  
}  
};
```

Why end descending for same start?

Longer interval should come first so shorter one can be detected as covered.

32. Common Edge Cases for Sorting Problems

32.1 Empty Input

Before accessing:

intervals[0]

check:

if (intervals.empty())

32.2 Single Element

Sorting still works.

Loop from index 1 should not run.

32.3 Duplicates

Important for:

- 3Sum
- 4Sum
- Largest Number
- Remove Covered Intervals
- Valid Anagram

32.4 Touching Intervals

Different problems define overlap differently.

For Merge Intervals:

{1,4}, {4,5}

should merge.

For Non-overlapping Intervals:

{1,2}, {2,3}

do not overlap.

For Arrows:

```
{1,2}, {2,3}
```

one arrow at 2 bursts both.

32.5 Original Indices

If needed, store:

```
vector<pair<int, int>> arr;  
  
for (int i = 0; i < nums.size(); i++) {  
    arr.push_back({nums[i], i});  
}
```

32.6 Integer Overflow in Comparator

Wrong:

```
return a[1] - b[1] < 0;
```

Can overflow.

Correct:

```
return a[1] < b[1];
```

32.7 Comparator Equality

Do not use <=.

Wrong:

```
return a[0] <= b[0];
```

Correct:

```
return a[0] < b[0];
```

32.8 All Zeroes in Largest Number

Input:

```
{0, 0}
```

Output should be:

```
"0"
```

not:

```
"00"
```

33. Common Mistakes and Wrong Code

Mistake 1 — Sorting When Order Matters

Wrong for stock prices:

```
sort(prices.begin(), prices.end());
```

Day order matters.

Mistake 2 — Wrong Sort Key

Merge intervals:

Sort by start.

Non-overlapping intervals:

Sort by end.

Queue reconstruction:

Height descending, k ascending.

Mistake 3 — Bad Comparator

Wrong:

```
return a[1] <= b[1];
```

Correct:

```
return a[1] < b[1];
```

Mistake 4 — Forgetting Duplicate Skips in 3Sum

Without duplicate skipping, result may contain repeated triplets.

Mistake 5 — Sorting and Losing Original Indices

If original index is required, preserve it.

Mistake 6 — Wrong Interval Endpoint Logic

Always clarify whether intervals are:

- closed [start, end]
- half-open [start, end)

Mistake 7 — Assuming Sorting Is Always Optimal

Hashing may be $O(n)$.

Heap may be $O(n \log k)$.

Quickselect may be average $O(n)$.

Sorting is simple but not always optimal.

34. Interview Explanation Templates

34.1 Sort and Scan

I sort the array so related values become adjacent. Then I scan once and compare neighbors. Sorting costs $O(n \log n)$, and the scan is $O(n)$.

34.2 Merge Intervals

I sort intervals by start time. Then I maintain a current merged interval. If the next interval overlaps, I extend the current interval. Otherwise, I push the current interval and start a new one.

34.3 Greedy by End Time

I sort intervals by end time because choosing the earliest ending interval leaves the most room for future intervals. Then I greedily keep intervals that do not overlap the last chosen interval.

34.4 Sort + Two Pointers

Sorting gives a predictable order. If the current sum is too small, I move the left pointer to increase it. If the sum is too large, I move the right pointer to decrease it.

34.5 Custom Comparator

Natural order is not enough, so I define a comparator based on the problem objective. For Largest Number, I compare $a+b$ and $b+a$ to decide which order creates the larger concatenation.

35. Complexity Patterns

Basic sorting	$O(n \log n)$	$O(\log n)$ typical
Sort + scan	$O(n \log n)$	$O(1)$ extra
Sort intervals + merge	$O(n \log n)$	$O(n)$ output
Sort intervals + greedy	$O(n \log n)$	$O(1)$ extra
Sort + two pointers	$O(n \log n)$ plus scan	$O(1)$
3Sum	$O(n^2)$	$O(1)$ extra
4Sum	$O(n^3)$	$O(1)$ extra
Sort + heap	$O(n \log n)$	$O(n)$
Custom string sort	$O(n \log n * m)$	depends

36. Time-Boxed Practice Guidance

Easy Sort + Scan

Target:

5-12 minutes

Examples:

- Contains Duplicate sorting version
 - Valid Anagram sorting version
 - Minimum Absolute Difference
-

Medium Interval / Greedy Sorting

Target:

20–35 minutes

Examples:

- Merge Intervals
 - Insert Interval
 - Non-overlapping Intervals
 - Minimum Number of Arrows
-

Harder Sorting + Custom Logic

Target:

30–50 minutes

Examples:

- 3Sum
 - 4Sum
 - Largest Number
 - Queue Reconstruction
 - Reorder Log Files
-

37. Expanded Practice Set

37.1 Must-Do Easy

- 1.1. Contains Duplicate, sorting version
 - 2.2. Valid Anagram, sorting version
 - 3.3. Assign Cookies
 - 4.4. Minimum Absolute Difference
 - 5.5. Squares of a Sorted Array
 - 6.6. Intersection of Two Arrays, sorting alternative
 - 7.7. Intersection of Two Arrays II, sorting alternative
 - 8.8. Reorder Data in Log Files
-

37.2 Must-Do Medium

- 9.9. Merge Intervals
- 10.10. Insert Interval
- 11.11. Non-overlapping Intervals
- 12.12. Meeting Rooms II
- 13.13. Minimum Number of Arrows to Burst Balloons
- 14.14. 3Sum
- 15.15. 3Sum Closest
- 16.16. 4Sum
- 17.17. K Closest Points to Origin

- 18.18. Largest Number
 - 19.19. Boats to Save People
 - 20.20. Queue Reconstruction by Height
 - 21.21. Remove Covered Intervals
 - 22.22. Sort Characters By Frequency
-

37.3 Strong Interview Practice

- 23.23. Car Fleet
 - 24.24. Hand of Straights
 - 25.25. Array of Doubled Pairs
 - 26.26. Reorganize String, heap overlap
 - 27.27. Task Scheduler, greedy/counting
 - 28.28. Top K Frequent Words
 - 29.29. Find K Closest Elements
 - 30.30. Russian Doll Envelopes
 - 31.31. Maximum Units on a Truck
 - 32.32. Frequency of the Most Frequent Element
-

37.4 Hard / Stretch

- 33.33. The Skyline Problem
 - 34.34. Russian Doll Envelopes
 - 35.35. Maximum Gap
 - 36.36. Count of Range Sum
 - 37.37. Count of Smaller Numbers After Self
 - 38.38. Reverse Pairs
-

38. Suggested Study Order

Step 1 — Basic Sort + Scan

- 23. Contains Duplicate
- 24. Valid Anagram
- 25. Minimum Absolute Difference

Goal:

Learn how sorting creates adjacency.

Step 2 — Interval Sorting

- 26. Merge Intervals
- 27. Insert Interval
- 28. Meeting Rooms
- 29. Non-overlapping Intervals
- 30. Minimum Number of Arrows

Goal:

Learn start-time vs end-time sorting.

Step 3 — Sort + Two Pointers

- 31. Two Sum II
- 32. 3Sum
- 33. 3Sum Closest

34. 4Sum

Goal:

Learn sorted sum logic and duplicate skipping.

Step 4 — Sorting + Greedy

- 35. Assign Cookies
- 36. Boats to Save People
- 37. Non-overlapping Intervals
- 38. Queue Reconstruction by Height

Goal:

Learn how ordering enables greedy choices.

Step 5 — Custom Comparator

- 39. K Closest Points
- 40. Largest Number
- 41. Reorder Data in Log Files
- 42. Sort Characters By Frequency

Goal:

Learn comparator design.

39. Pattern Decision Table

duplicates	value ascending	compare adjacent
anagram	characters ascending	compare strings
merge intervals	start ascending	scan and merge
meeting conflict	start ascending	compare adjacent
min removals for intervals	end ascending	greedy keep
min arrows	end ascending	greedy coverage
pair sum	value ascending	two pointers
3Sum / 4Sum	value ascending	fix + two pointers
closest points	distance ascending	take first k
largest number	$a+b > b+a$	concatenate
assign resources	small to large	greedy
queue reconstruction	height desc, k asc	insert by k
logs	custom rules	stable sort

40. Final Cheat Sheet

Basic Sort

```
sort(nums.begin(), nums.end());
```

Descending Sort

```
sort(nums.begin(), nums.end(), greater<int>());
```

Sort Intervals by Start

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    return a[0] < b[0];  
});
```

Sort Intervals by End

```
sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>& b) {  
    return a[1] < b[1];  
});
```

Merge Intervals

```
sort(intervals.begin(), intervals.end());  
  
vector<vector<int>>> result;  
vector<int> current = intervals[0];  
  
for (int i = 1; i < intervals.size(); i++) {  
    if (intervals[i][0] <= current[1]) {  
        current[1] = max(current[1], intervals[i][1]);  
    } else {  
        result.push_back(current);  
        current = intervals[i];  
    }  
}  
  
result.push_back(current);
```

Greedy by End

```

sort(intervals.begin(), intervals.end(), [](const vector<int>& a, const vector<int>&
b) {
    return a[1] < b[1];
});

int lastEnd = intervals[0][1];
int removals = 0;

for (int i = 1; i < intervals.size(); i++) {
    if (intervals[i][0] < lastEnd) {
        removals++;
    } else {
        lastEnd = intervals[i][1];
    }
}

```

Sort + Two Pointers

```

sort(nums.begin(), nums.end());

int left = 0;
int right = nums.size() - 1;

while (left < right) {
    int sum = nums[left] + nums[right];

    if (sum < target) {
        left++;
    } else if (sum > target) {
        right--;
    } else {
        // found
        left++;
        right--;
    }
}

```

Largest Number Comparator

```

sort(words.begin(), words.end(), [](const string& a, const string& b) {
    return a + b > b + a;
});

```

Queue Reconstruction Comparator

```
sort(people.begin(), people.end(), [](const vector<int>& a, const vector<int>& b) {
    if (a[0] != b[0]) {
        return a[0] > b[0];
    }

    return a[1] < b[1];
});
```

41. What to Master Before Chapter 8

Before moving to Custom Comparator, make sure you can:

- Use `sort(nums.begin(), nums.end())`
 - Sort descending
 - Sort pairs and vectors
 - Sort intervals by start
 - Sort intervals by end
 - Know when to sort by start vs end
 - Merge intervals
 - Apply greedy by earliest end
 - Use sort + two pointers
 - Skip duplicates in 3Sum
 - Preserve original indices when needed
 - Avoid sorting when original order matters
 - Write simple lambda comparators
 - Understand strict comparator rules
 - Analyze $O(n \log n)$ sorting complexity
-

42. Core Takeaway

Sorting is not just arranging values.

Sorting creates structure.

After sorting:

- duplicates become adjacent
- intervals can be merged
- greedy choices become clearer
- two pointers become possible
- closest values become neighbors
- custom ordering can directly optimize the result

The main question is:

If I sort the data in the right way, does the problem become a scan, two-pointer problem, greedy problem, or interval problem?

If yes, sorting is probably the key.

The biggest warning:

Do not sort if original order, original indices, subarray structure, or time order matters.
